



가천대학교 스마트시티학과 – 창업프로젝트

Lecture 01 - Introduction

Chris Ohk

utilForever@gmail.com

- 옥찬호 (Chris Ohk)
 - (현) Hanyang University Visiting Senior Researcher
 - (전) 42dot Senior Embedded Software Engineer
 - (전) EJN Tech Lead
 - (전) Momenti Engine Engineer
 - (전) Nexon Korea Game Programmer
 - Microsoft Developer Technologies MVP
 - C++ Korea Founder & Administrator
 - Reinforcement Learning KR Administrator
 - IT 전문서 집필 및 번역 다수

utilForever@gmail.com



utilForever



- 강의 개요

- 스마트시티에 존재하는 문제들을 발굴하고, 자료 조사를 통해 아이디어를 검증한다.
이후 Python 기반 웹 프레임워크, 데이터베이스, 공공데이터 및 외부 API를 활용하여 SaaS 형태의 MVP를 개발하고 클라우드 환경에 배포까지의 과정을 학습한다.
또한 실제 사용자 테스트와 서비스 지표를 기반으로 제품을 개선한다.

- 강의 목표

- 스마트시티 관련 문제들을 발굴하고 아이디어를 검증할 수 있는 역량 습득
- SaaS 형태의 MVP를 개발하고 배포할 수 있는 역량 습득
- 실제 사용자 테스트와 피드백을 통해 서비스를 개선할 수 있는 역량 습득
- 원하는 아이디어를 기반으로 서비스를 직접 구현하는 프로젝트 수행

- 도시 문제 발견과 웹 서비스 기초 학습
 - SaaS와 MVP 이해
 - 도시 문제 관찰과 서비스 기회 발견
 - 사용자, 고객 및 도시 이해관계자 정의
 - 사용자 인터뷰와 문제 검증
 - 클라이언트, 서버, URL, HTTP 및 웹 애플리케이션 이해
 - Django를 이용한 기본 웹 애플리케이션 개발

- MVP 설계와 초기 서비스 개발
 - 기존 해결 방법과 경쟁 서비스 분석
 - 가치 제안과 사업 모델 정의 / MVP 핵심 기능 선정
 - 사용자 스토리와 제품 요구사항 작성
 - 사용자 흐름과 화면 프로토타입 설계
 - Django Template과 Bootstrap을 이용한 웹 화면 제작
 - Django Model을 이용한 데이터 모델링
 - CRUD와 사용자 입력 기능 구현
 - 사용자 인증, 권한 및 사용자별 데이터 구현
 - 초기 프로젝트 프로토타입 통합

- 핵심 서비스 완성과 배포
 - 공공데이터와 외부 API 연동
 - CSV, JSON 및 공간 데이터 처리
 - 핵심 사용자 흐름의 End-to-End 구현
 - 사용자 입력, 오류 및 외부 API 장애 처리
 - 자동화 테스트 작성 / 코드 리뷰와 리팩터링
 - 지속적 통합 적용 / 애플리케이션과 데이터베이스 클라우드 배포
 - 환경 변수와 민감정보 관리
 - 로그, 모니터링, 보안 및 백업 구성

- 사용자 검증과 제품 개선
 - 공개 베타 버전 배포
 - 사용자 인터뷰와 사용성 테스트
 - 사용자 피드백과 서비스 이용 데이터 수집
 - 핵심 제품 지표 정의와 분석
 - 버그, 기능 요청 및 개선사항 우선순위 결정
 - 사용자 증거를 바탕으로 제품 개선
 - 사업 모델과 서비스 운영 계획 보완
 - 최종 서비스 시연과 발표 준비
 - 최종 발표 및 최종 보고서 제출

- 전제 조건
 - Python Programming
 - Git & GitHub
- 교재
 - 없음
- 평가 배점
 - 중간고사, 기말고사가 없는 매우 행복한 과목
 - 출석 15% + 계획 보고서 15% + 중간 발표 20% + 최종 발표 30% + 최종 보고서 20%

- 출석 (15%)
 - 강의 시작 후 15분까지는 출석 인정
 - 강의 시작 후 15~30분까지는 지각 처리
 - 강의 시작 후 30분이 넘어서 온 경우 결석 처리
 - 특별한 사유가 있을 경우 미리 알려줄 것
 - 결석 2번(또는 지각 4번)까지는 감점 없음
 - 지각 1번당 **0.5점** 감점
 - 결석 1번당 **1점** 감점

- 계획 보고서 (15%)
 - 개인 프로젝트로 진행
 - 스마트시티 주제로 만들고 싶은 서비스를 정해 계획 문서를 제출
 - 자세한 내용은 2주차 수업 때 안내

- 중간 발표 (20%)
 - 서비스를 완성할 필요 없음
 - 처음에 계획 보고서에 냈던 방향으로 잘 진행하고 있는지 확인하기 위함
 - 자세한 내용은 5주차 수업 때 안내

- 최종 발표 (30%)
 - 서비스를 완성해서 발표해야 함
 - 만든 서비스를 시연하고 질문 및 답변을 진행
 - 자세한 내용은 10주차 수업 때 안내
- 최종 보고서 (20%)
 - 서비스를 어떻게 설계했고, 데이터를 어떻게 사용했는지, 기술적 어려움이 있었는지, 사용자 피드백을 반영했는지 등의 내용으로 구성
 - 최종 발표 이후 제출하면 됨
 - 자세한 내용은 10주차 수업 때 안내

- LLM(Large Language Model) 사용 관련
 - 공부를 하고 과제를 하는 과정에서 LLM을 자유롭게 사용할 수 있습니다.
 - 저는 여러분들이 제출한 보고서, 계획서, 그리고 발표와 질문 / 답변 과정에서 어떤 생각으로 설계했는지, 코드를 잘 이해하고 있는지 등을 확인할 겁니다.
 - LLM이 항상 정답을 출력할 것이라고 생각하지 마세요.
 - 본인이 생각을 주도하고 LLM에게 도움을 받아야 합니다.
LLM에게 자신이 생각해야 될 것들을 모두 떠넘기지 마세요.

Software as a Service (SaaS)

창업프로젝트
Introduction

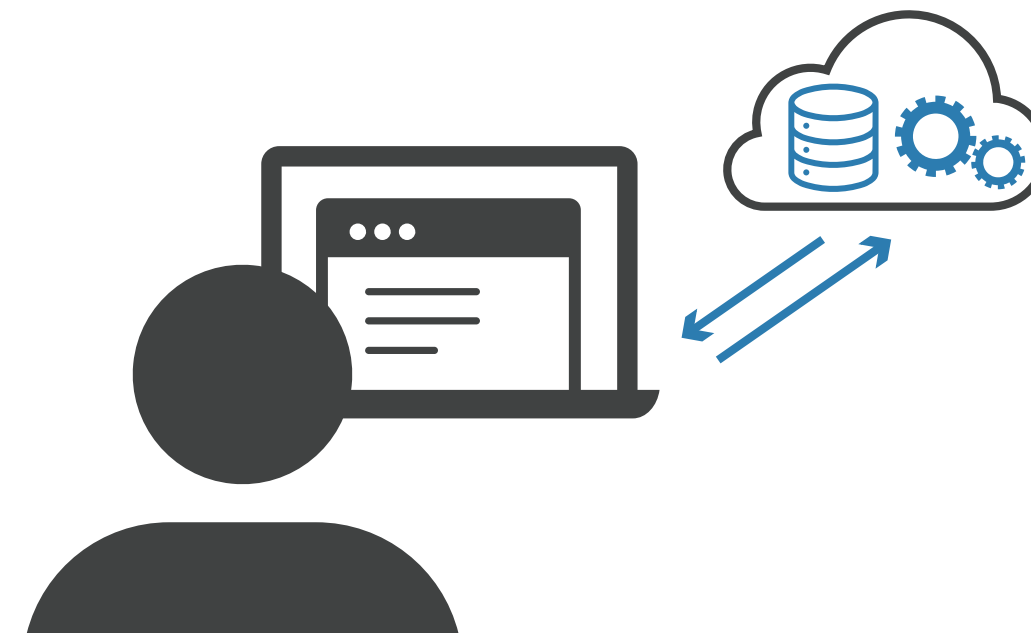
A cloud computing model where software is **delivered over the internet** and **accessed via a web browser** instead of being installed locally on a computer

Non-SaaS Application



Application logic runs
on user's computer

SaaS Application

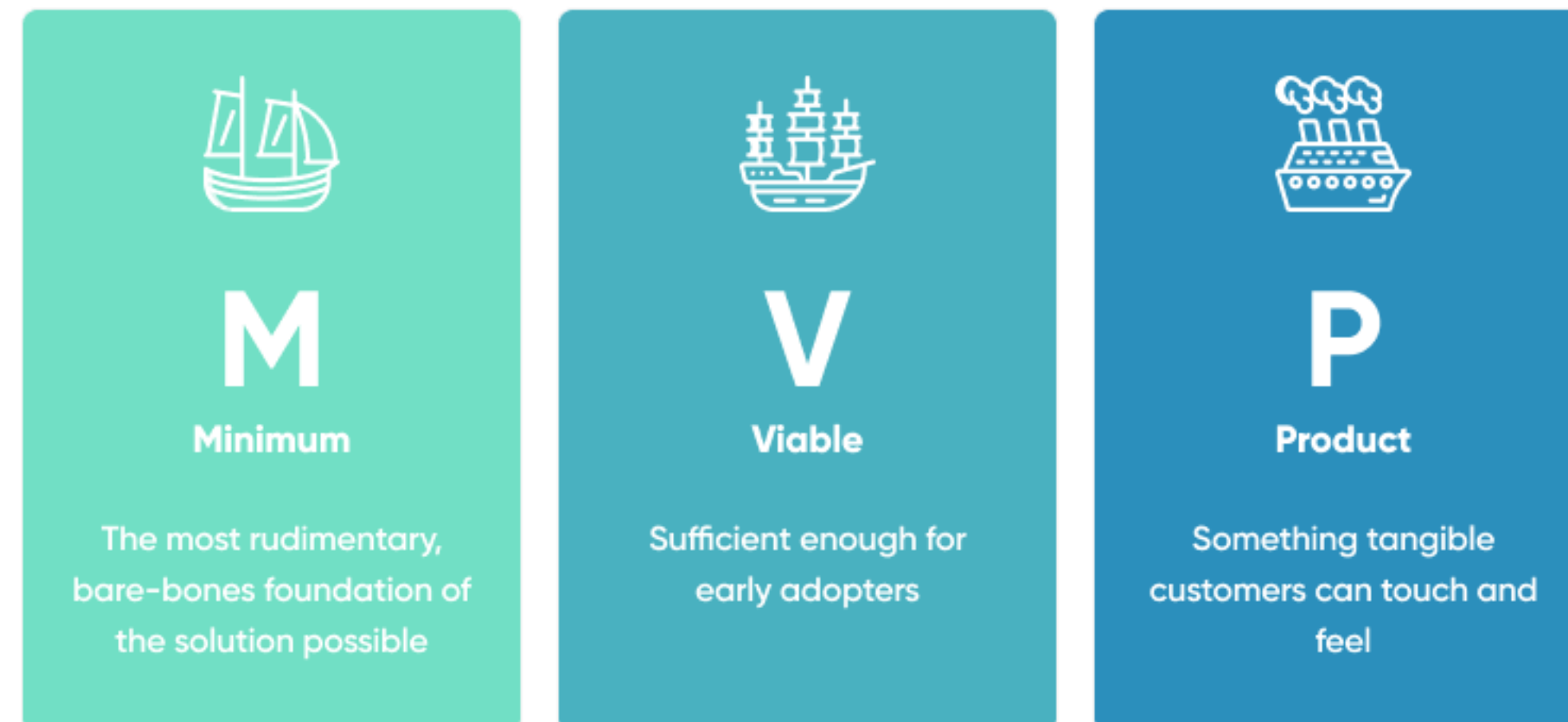


Application logic runs
in the cloud

Minimum Viable Product (MVP)

창업프로젝트
Introduction

A product development strategy where a new product is built with the minimum set of features required to **deliver value to early users** and **validate a business idea**



How to Build a Minimal Viable Product (MVP)
(<https://adamfard.com/blog/build-mvp>)

Minimum Viable Product (MVP)

창업프로젝트
Introduction

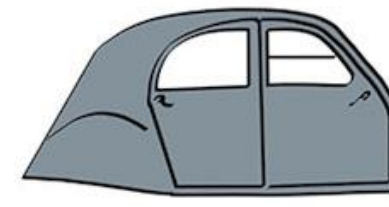
HOW **NOT TO BUILD** A MINIMUM VIABLE PRODUCT



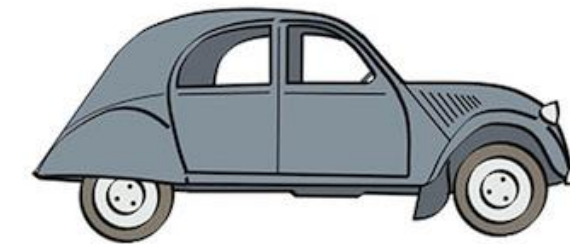
1



2



3



4

ALSO HOW **NOT TO BUILD** A MINIMUM VIABLE PRODUCT



1



2



3

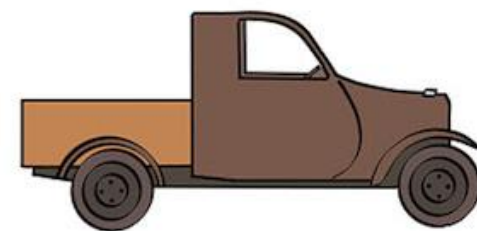


4

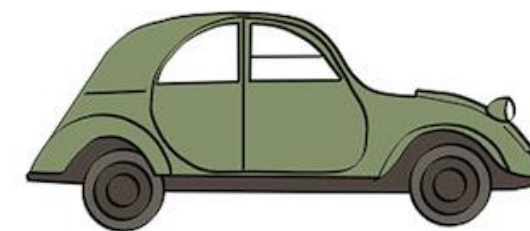
HOW **TO BUILD** A MINIMUM VIABLE PRODUCT



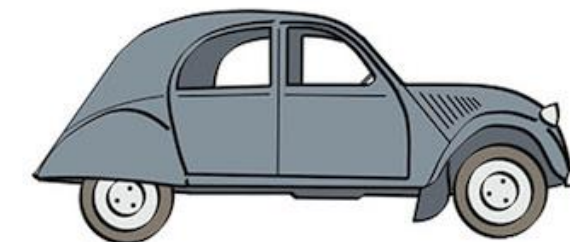
1



2



3



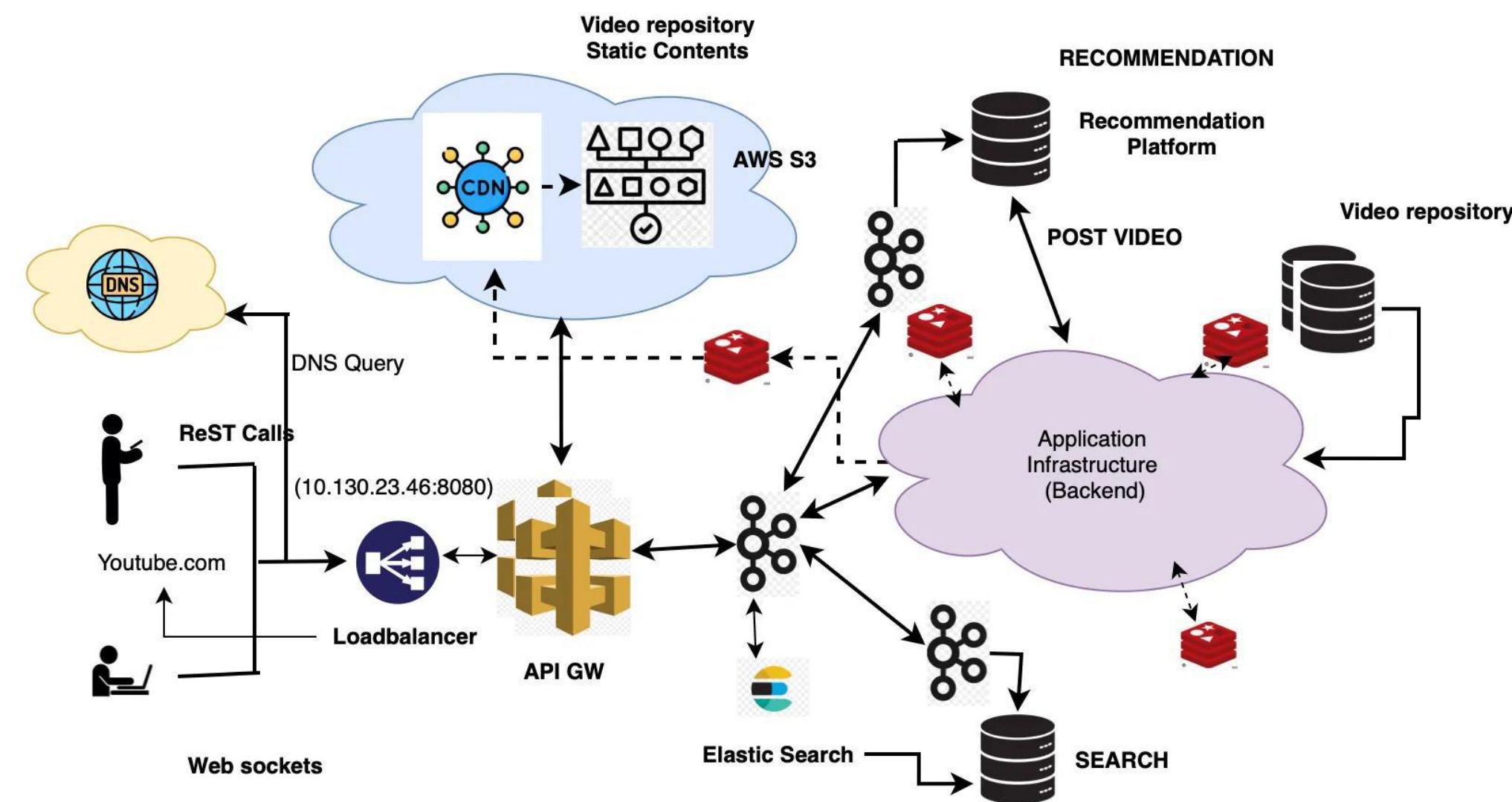
4

FRED VOORHORST

EDUCATI.CH

MVP – Really going from bike to car?
(<https://www.educati.ch/mvp-really-going-bike-car/>)

The process of defining the architecture, components, modules, interfaces, and data for a system to **satisfy specified requirements.**



- 시스템 설계는 정답이 없습니다
 - 여러 가지 방법이 있고, 그 중에서 최선을 선택해야 합니다.
 - 많이 생각하고, 많이 질문하고, 많이 논의하는 게 좋습니다.
 - 다양한 실제 사례들을 가져와 어떻게 / 왜 이렇게 설계했는지 분석해 보세요.
 - 예를 들어, 여러분이 파일 업로드 시스템을 만들어야 된다고 가정해 봅시다.
 - 일반적인 환경이라면 네트워크가 안정적일 것이다. 하지만 네트워크가 불안정한 환경이라면 어떻게 해야 할까?
 - 일반적인 환경이라면 사양에 큰 제약이 없을 것이다. 하지만 임베디드에서 서비스해야 한다면 어떻게 해야 할까?
 - 일반적인 환경이라면 디스크를 사용할 수 있을 것이다. 하지만 디스크를 사용할 수 없다면 어떻게 해야 할까?
 - 데이터 발생 주기가 매우 짧다면 어떻게 해야 할까?
 - ...

Example

Coduck - CP Editorial

Search

Home

Categories

Upload Guide

Copyright

☀

Language

English ▾

Find Contest Editorials

Search competitions and quickly open or download editorials by keyword.

Start with a keyword

Search

Browse categories



Editorials

1310

Competitions

200

Index Version

2026.09.01

What is CP Editorial?

CP Editorial is a curated archive for competitive-programming contest editorials.

It helps you find contest write-ups quickly and access original editorial files from cp-editorial-data.

- Browse by category and contest hierarchy.
- Search by contest name, year, or editorial title.
- Open or download editorial source files directly.

CP Editorial – A curated archive for competitive-programming contest editorials
(<https://editorial.coduck.io/>)

감사합니다.

utilForever@gmail.com

<https://github.com/utilForever>

Facebook, Twitter, Instagram, LinkedIn: utilForever